

RangeGraph

Augmented Dynamic Graph Maintenance via Lazy Segment Trees and Binary Indexed Trees

Based on: D-Tree (VLDB 2022) · DS-Tree (SIGMOD 2024)

Extending with: Lazy Segment Trees · BIT · Lexicographic Edge Indexing

Samiul Farhan Sumel Joy

2616143650

Summer-26

CSE-511

Section: 1

Motivation — Why This Problem Matters

REAL-WORLD SCENARIO

Dhaka City Road Network

5,000+ intersections (nodes)
12,000+ road segments (edges)

Every minute:

- Roads close due to floods
- New detours open
- Traffic weights update
- Queries: Is route A→B open?
- Queries: Cheapest path in zone 3?

What existing systems can't do

D-Tree / DS-Tree answer:

✓ Is node A connected to B?

But cannot answer:

- ✗ Update ALL edge weights in zone 3 by +5 (one step)
- ✗ Find min-cost edge in range [node 100 – 200]
- ✗ K-hop neighbours efficiently

Our answer: Augment dynamic graph structures with a lazy segment tree layer — enabling $O(\log n)$ bulk updates, range edge queries, and k-hop lookups on fully dynamic graphs.

Literature Review

Paper / Year	Venue	Tools / Structures	What It Solved	Shortcomings
D-Tree (Chen et al., 2022)	VLDB 2022	Spanning trees, union-find, adjacency lists	Fast dynamic connectivity on large real graphs; avg-case faster than worst-case bounds	No range queries; no bulk updates; no lexicographic edge order; no k-hop support
DS-Tree (Xu et al., 2024)	SIGMOD 2024	Dual spanning tree + disjoint-set (union-find)	$O(1)$ connectivity queries; best-known query time for dynamic undirected graphs	Higher update cost; still no range query layer; single-edge operations only
Dynamic Seg. Tree (IEEE Trans., 2003)	IEEE Trans. Computers	Balanced BST augmented with range sets, endpoint scheme	Dynamic range insert/delete/query without knowing all keys in advance	Not applied to graphs; no lazy propagation; no edge-pair ordering
BITS-Tree (Easwarakumar, 2015)	arXiv	Modified segment tree with extended interval nodes	Faster dynamic insertion/deletion of segments; faster stabbing & range queries than standard seg tree	Not graph-aware; no lazy bulk update support; no integration with connectivity structures
Concurrent Dyn. Conn. (Fedorov et al., 2021)	SPAA 2021	Euler Tour Trees, concurrent spanning forest	First truly concurrent dynamic connectivity with sequential-equivalent time complexity	Complex implementation; no range query primitives; not benchmarked on standard graph datasets

The Research Gap

No existing dynamic graph structure integrates range query primitives at the edge/adjacency level.

Q1 Bulk increment updates

Update edge weights by $\Delta > 1$ across a range in one operation — not one edge at a time.

Q2 Efficient insert & delete

Insert or delete graph edges in $O(\log n)$ without rebuilding any structure from scratch.

Q3 Lexicographic edge ordering

Sort and query (u,v) edge pairs in dictionary order for efficient range scans.

Q4 Adj. list + matrix + $N(k)$

Support both graph representations and answer k -hop neighbourhood queries via range queries.

Q5 Dynamic graph maintenance

Maintain connectivity and range structures simultaneously as edges appear and disappear.

Proposed Approach — System Architecture

Graph Input Layer

Dynamic edge inserts / deletes / weight updates



Augmented D-Tree Core

Spanning tree per component + union-find for $O(1)$ connectivity



Lazy Segment Tree + BIT Layer

Lexicographic (u,v) edge indexing · bulk Δ -updates · range queries · k-hop $N(k)$

Key Techniques

BIT

Fenwick Tree

Prefix sum queries in $O(\log n)$

SEG

Lazy Segment Tree

Bulk range updates, deferred

LEX

Lex. Edge Ordering

Binary search over (u,v) pairs

DYN

Dynamic Structure

Insert/delete without rebuild

Expected gain: $O(n)$ bulk updates $\rightarrow O(\log n)$ · Range queries not possible $\rightarrow O(\log n)$ · k-hop $N(k) \rightarrow O(k \log n)$

Datasets for Benchmarking

SNAP Road Networks

Stanford SNAP · ~1M nodes / 2.7M edges · Real-world

Dynamic edge changes simulated by timestamped road events

DIMACS Challenge Graphs

DIMACS · 10K – 10M nodes · Benchmark

Standard graphs used in every dynamic connectivity paper; enables direct comparison

DBLP Co-authorship

SNAP · ~317K nodes / 1M edges · Real-world

Social/collaboration graph; dynamic edge insertions as new papers are published

LiveJournal Social

SNAP · Stress test

Stress-tests scalability and bulk update performance at social-network scale

Synthetic Random Graphs

Generated (Erdős–Rényi) · Controlled

Controlled experiments isolating specific parameters (density, update rate, range width)

10-Week Project Plan

Biweekly professor check-ins at weeks 2 · 4 · 6 · 8 · 10

Phase	Tasks	Checkpoint Deliverable
Wk 1–2	Baseline Reproduction Read D-Tree & DS-Tree · Clone D-Tree GitHub code · Run on SNAP dataset · Document weaknesses	✓ Baseline running + weakness report
Wk 3–4	Range Query Layer Build segment tree with lazy propagation · Test independently · Design edge-pair key scheme (u,v) lexicographic	✓ Standalone seg tree tested
Wk 5–6	Integration Augment D-Tree with segment tree · Sync on every insert/delete · Implement range edge queries · Lexicographic ordering	✓ Integrated prototype working
Wk 7–8	Full Feature Set Lazy bulk Δ -updates (Q1) · k-hop N(k) via range queries (Q4) · 2D BIT for adjacency matrix · All 5 RQs answered in code	✓ All 5 research questions answered
Wk 9–10	Benchmarking + Paper Experiments vs D-Tree / DS-Tree on SNAP + DIMACS · Plot results · Write IEEE-format paper · Prepare final slides	✓ Paper + benchmarks + slides submitted

Thank You

Project: RangeGraph — Dynamic Graph + Range Query Integration

Baseline: D-Tree (VLDB 2022) + DS-Tree (SIGMOD 2024)

Contribution: Lazy segment tree + BIT layer on dynamic graph adjacency

Gain: $O(\log n)$ bulk updates, range edge queries, k-hop $N(k)$

Duration: 10 weeks · 5 biweekly check-ins · IEEE-format paper

Questions & Discussion

Datasets: Stanford SNAP · DIMACS · Code: github.com/qingchen3/D-tree · Paper draft available on request